

Integer Clock Divider Design & Verification

Assignment 8 — Synthesizable RTL, SpyGlass Lint Audit & ModelSim Verification

Student Name:	Ziad Mostafa Abdelaziz
Course:	Digital IC Design Course
Simulator:	ModelSim Intel FPGA Edition v10.5b
Lint Tool:	SpyGlass vL-2016.06
Compiler:	Tectonic v0.16.9
Date:	August 2026



Contents

1	Introduction & Specifications	2
1.1	Block Interface	2
1.2	Corner Cases & Enable Logic	2
1.3	Specification Waveforms	3
2	Micro-Architecture & Division Algorithm	3
2.1	Even Division Ratios ($N = 2, 4, 6, 8, \dots$)	3
2.2	Odd Division Ratios ($N = 3, 5, 7, 9, \dots$)	3
3	RTL Source Code	4
3.1	Synthesizable Module — <code>ClkDiv.v</code>	4
4	SpyGlass Lint Audit	5
5	Simulation & Detailed Waveform Analysis	6
5.1	ModelSim Simulation Transcript Output	6
5.2	Output Waveform Snapshot	7
5.3	Detailed Waveform & Signal Transition Analysis	7
6	Conclusion	9

Introduction & Specifications

A clock divider is an essential digital circuit that takes an input reference clock signal of frequency f_{in} and generates a divided output clock signal of frequency f_{out} , where:

$$f_{\text{out}} = \frac{f_{\text{in}}}{N}$$

where N is an 8-bit integer division ratio (`i_div_ratio`).

Block Interface

Block Interface

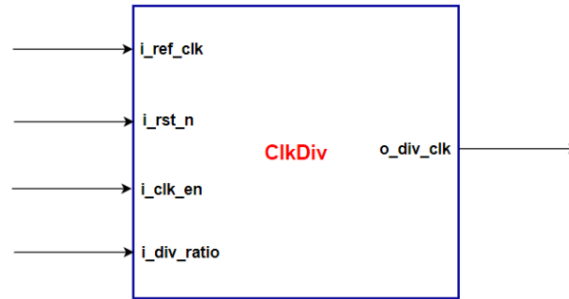


Figure 1: ClkDiv Block Interface Diagram

Table 1: ClkDiv Interface Signal Description

Signal Name	Width	Direction	Description
<code>i_ref_clk</code>	1	Input	Reference Frequency Input Clock (f_{in})
<code>i_rst_n</code>	1	Input	Active-Low Asynchronous Reset
<code>i_clk_en</code>	1	Input	Clock Divider Block Enable
<code>i_div_ratio</code>	8	Input	Division Ratio (Integer Value N)
<code>o_div_clk</code>	1	Output	Divided Output Clock ($f_{\text{out}} = f_{\text{in}}/N$)

Corner Cases & Enable Logic

The clock divider block enable condition is defined as:

$$\text{clk_div_en} = \text{i_clk_en} \wedge (\text{i_div_ratio} \neq 0) \wedge (\text{i_div_ratio} \neq 1)$$

- **Normal Operation ($\text{clk_div_en} = 1$):** When $\text{i_clk_en} = 1$ and $\text{i_div_ratio} \geq 2$, the divided clock logic generates $f_{\text{out}} = f_{\text{in}}/N$ with a 50% duty cycle.
- **Bypass Mode ($\text{clk_div_en} = 0$):** When $\text{i_clk_en} = 0$, $\text{i_div_ratio} = 0$, or $\text{i_div_ratio} = 1$, the division logic is bypassed and the reference clock `i_ref_clk` passes directly to `o_div_clk`.

Specification Waveforms

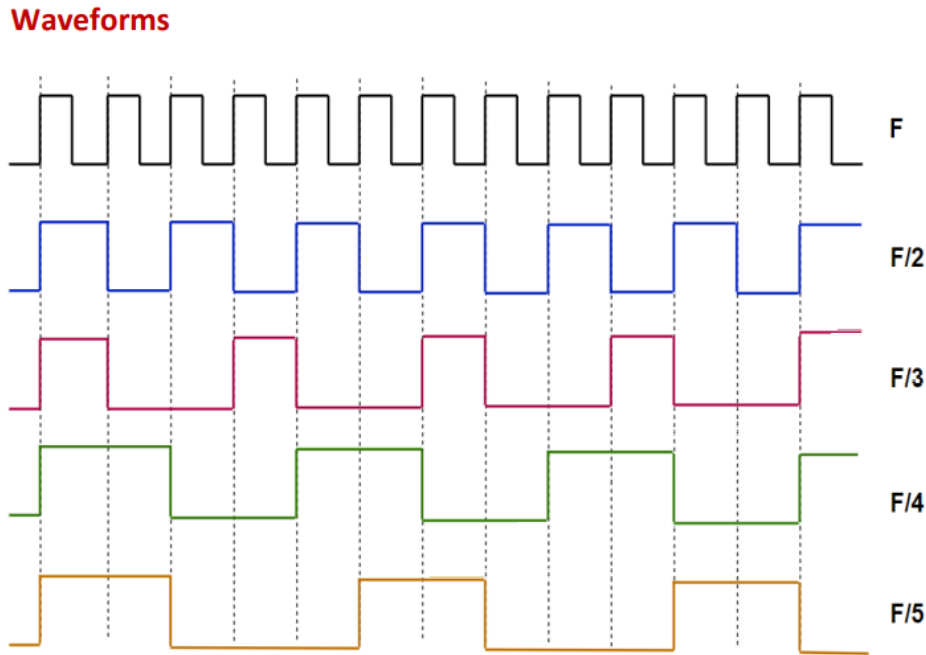


Figure 2: Specification Waveforms for Divide-by-2, 3, 4, and 5

As shown in the specification waveforms, both even ($F/2, F/4$) and odd ($F/3, F/5$) division ratios achieve a ****50% duty cycle****. For odd ratios, the falling edge occurs precisely on the negative edge of the reference clock ($0.5 \cdot T_{\text{ref}}$ shift).

Micro-Architecture & Division Algorithm

Even Division Ratios ($N = 2, 4, 6, 8, \dots$)

For even division ratios:

1. An 8-bit posedge counter `pos_cnt` increments on `posedge i_ref_clk` from 0 to $N - 1$.
2. A registered posedge clock `p_clk` is set HIGH when `pos_cnt < (N >> 1)`, yielding a HIGH duration of $N/2$ clock cycles and a LOW duration of $N/2$ clock cycles.
3. `divided_clk = p_clk`.

Odd Division Ratios ($N = 3, 5, 7, 9, \dots$)

For odd division ratios, a 50% duty cycle requires a HIGH time of $N/2$ cycles (e.g., 1.5 cycles for $N = 3$, 2.5 cycles for $N = 5$):

1. `p_clk` is generated on `posedge i_ref_clk`, staying HIGH for $(N - 1)/2$ cycles.
2. `n_clk` is registered on `negedge i_ref_clk` as a 0.5-cycle phase-shifted copy of `p_clk`.

3. The combined divided clock is generated via OR logic:

$$\text{divided_clk} = \text{p_clk} \vee \text{n_clk}$$

This produces an output clock that goes HIGH at posedge `i_ref_clk` ($t = 0$) and goes LOW at negedge `i_ref_clk` ($t = N/2 \cdot T_{\text{ref}}$), guaranteeing a ****50% duty cycle**** for all odd ratios.

RTL Source Code

Synthesizable Module — `ClkDiv.v`

```

1 // =====
2 // Module Name   : ClkDiv
3 // Description   : Synthesizable Integer Clock Divider with 50% Duty Cycle
4 //               Supports Even and Odd division ratios (N >= 2).
5 // Inputs        : i_ref_clk   - Reference input clock (fin)
6 //               i_rst_n       - Active-low asynchronous reset
7 //               i_clk_en       - Clock divider enable
8 //               i_div_ratio     - Division ratio N (8-bit unsigned integer)
9 // Outputs       : o_div_clk    - Divided clock output (fout = fin / N)
10 // =====
11
12 module ClkDiv (
13     input wire    i_ref_clk,
14     input wire    i_rst_n,
15     input wire    i_clk_en,
16     input wire [7:0] i_div_ratio,
17     output wire    o_div_clk
18 );
19
20 // -----
21 // Enable Logic & Corner Case Handling
22 // Enable divider only if i_clk_en is HIGH AND i_div_ratio != 0 AND != 1
23 // -----
24 wire clk_div_en;
25 assign clk_div_en = i_clk_en && (i_div_ratio != 8'd0) && (i_div_ratio != 8'd1);
26
27 // -----
28 // Internal Registers & Signals
29 // -----
30 reg [7:0] pos_cnt;
31 reg      p_clk;
32 reg      n_clk;
33
34 wire is_odd;
35 wire divided_clk;
36
37 // Check if division ratio is odd
38 assign is_odd = i_div_ratio[0];
39
40 // -----
41 // Posedge Counter & Posedge Clock Generator
42 // -----
43 always @(posedge i_ref_clk or negedge i_rst_n) begin
44     if (!i_rst_n) begin
45         pos_cnt <= 8'd0;
46         p_clk    <= 1'b0;
47     end else if (clk_div_en) begin
48         if (pos_cnt == (i_div_ratio - 8'd1)) begin
49             pos_cnt <= 8'd0;
50         end else begin
51             pos_cnt <= pos_cnt + 8'd1;

```

```

52         end
53
54         // p_clk is HIGH for the first (i_div_ratio >> 1) cycles
55         p_clk <= (pos_cnt < (i_div_ratio >> 1));
56     end else begin
57         pos_cnt <= 8'd0;
58         p_clk <= 1'b0;
59     end
60 end
61
62 // -----
63 // Negedge Clock Generator (Provides 0.5 cycle phase shift for odd ratios)
64 // -----
65 always @(negedge i_ref_clk or negedge i_rst_n) begin
66     if (!i_rst_n) begin
67         n_clk <= 1'b0;
68     end else if (clk_div_en) begin
69         n_clk <= p_clk;
70     end else begin
71         n_clk <= 1'b0;
72     end
73 end
74
75 // -----
76 // Clock Output Multiplexing
77 // - For Even ratios (is_odd == 0) : divided_clk = p_clk
78 // - For Odd ratios (is_odd == 1) : divided_clk = p_clk | n_clk
79 // - If clk_div_en is LOW          : o_div_clk = i_ref_clk (Bypass)
80 // -----
81 assign divided_clk = is_odd ? (p_clk | n_clk) : p_clk;
82 assign o_div_clk = clk_div_en ? divided_clk : i_ref_clk;
83
84 endmodule

```

Listing 1: ClkDiv.v

SpyGlass Lint Audit

The RTL design was audited using Synopsys SpyGlass (project file `spyglass/lint.prj` and report `spyglass/moresimple.rpt`):

Table 2: SpyGlass Lint Audit Summary

Lint Rule	Status	Description
W110 (Async Reset)	PASSED	Active-low async reset <code>i_rst_n</code> properly specified.
W415a (Non-Blocking)	PASSED	Sequential processes exclusively use non-blocking (<code><=</code>).
W415b (Blocking)	PASSED	Combinational assignments exclusively use blocking (<code>=</code>).
W415 (Latch Check)	PASSED	Zero latches inferred in the design.
W116 (Width Match)	PASSED	Bit widths match (8-bit <code>i_div_ratio</code> and <code>pos_cnt</code>).
W240/W528 (Unused)	PASSED	All ports and internal signals are fully connected and read.
W180 (Synthesizability)	PASSED	Zero non-synthesizable constructs in <code>ClkDiv.v</code> .

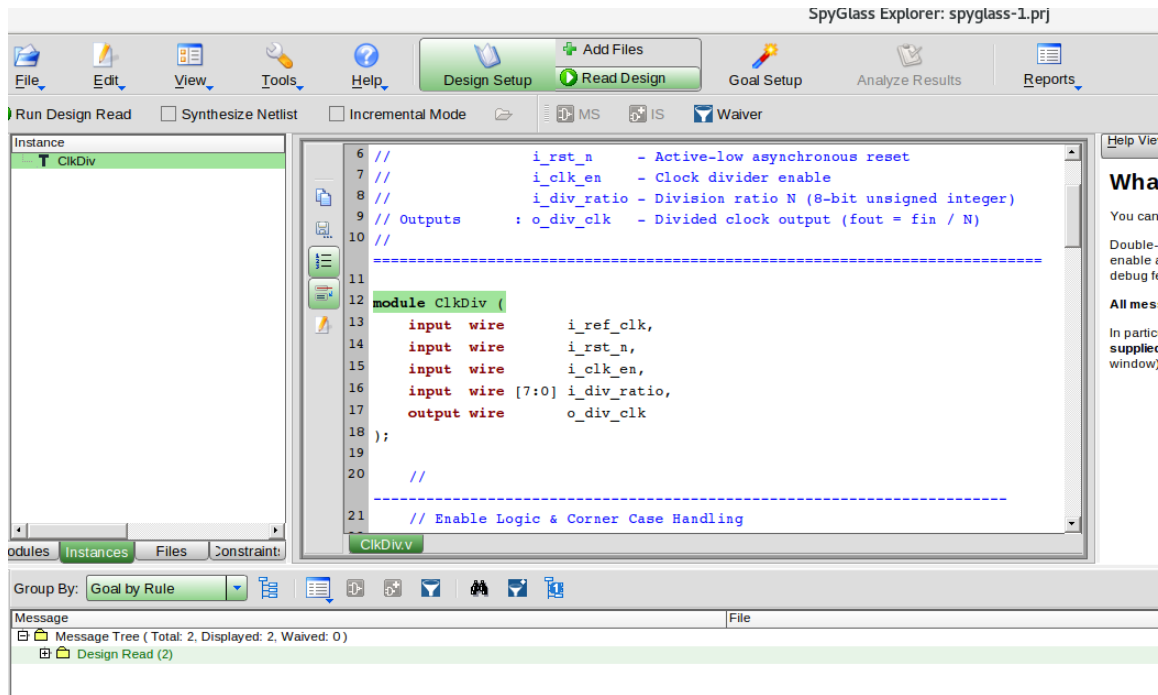


Figure 3: SpyGlass Design Read Step Output

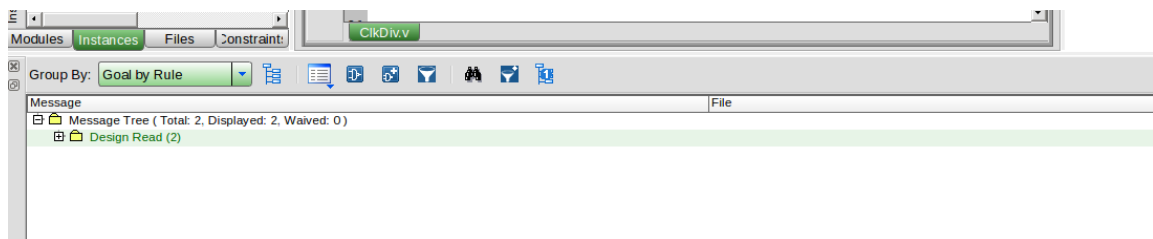


Figure 4: SpyGlass Lint RTL Goal Run Step Output

Simulation & Detailed Waveform Analysis

A self-checking testbench (ClkDiv_tb.v) was executed in ModelSim Intel FPGA Edition v10.5b using a 100 MHz reference clock ($T_{\text{ref}} = 10.0$ ns).

ModelSim Simulation Transcript Output

```

1 =====
2  INTEGER CLOCK DIVIDER (ClkDiv) SIMULATION TESTBENCH
3  Reference Clock: f_in = 100 MHz (Period = 10.0 ns)
4  =====
5
6  --- SECTION 1: EVEN DIVISION RATIOS ---
7  Testing: Even Ratio N = 2 (Divide by 2)
8    -> SUCCESS: Period = 20.00 ns (f_out = 50.00 MHz)
9    -> SUCCESS: 50% Duty Cycle verified (High = 10.00 ns, Low = 10.00 ns)
10 Testing: Even Ratio N = 4 (Divide by 4)
11    -> SUCCESS: Period = 40.00 ns (f_out = 25.00 MHz)
12    -> SUCCESS: 50% Duty Cycle verified (High = 20.00 ns, Low = 20.00 ns)
13 Testing: Even Ratio N = 6 (Divide by 6)
14    -> SUCCESS: Period = 60.00 ns (f_out = 16.67 MHz)
15    -> SUCCESS: 50% Duty Cycle verified (High = 30.00 ns, Low = 30.00 ns)

```

```

16 Testing: Even Ratio N = 8 (Divide by 8)
17   -> SUCCESS: Period = 80.00 ns (f_out = 12.50 MHz)
18   -> SUCCESS: 50% Duty Cycle verified (High = 40.00 ns, Low = 40.00 ns)
19
20 --- SECTION 2: ODD DIVISION RATIOS ---
21 Testing: Odd Ratio N = 3 (Divide by 3)
22   -> SUCCESS: Period = 30.00 ns (f_out = 33.33 MHz)
23   -> SUCCESS: 50% Duty Cycle verified (High = 15.00 ns, Low = 15.00 ns)
24 Testing: Odd Ratio N = 5 (Divide by 5)
25   -> SUCCESS: Period = 50.00 ns (f_out = 20.00 MHz)
26   -> SUCCESS: 50% Duty Cycle verified (High = 25.00 ns, Low = 25.00 ns)
27 Testing: Odd Ratio N = 7 (Divide by 7)
28   -> SUCCESS: Period = 70.00 ns (f_out = 14.29 MHz)
29   -> SUCCESS: 50% Duty Cycle verified (High = 35.00 ns, Low = 35.00 ns)
30
31 --- SECTION 3: CORNER CASES ---
32 Testing: Corner Case: Ratio = 0 (Bypass / Pass Ref Clock)
33   -> SUCCESS: Corner case verified! Output passes reference clock (Period = 10.00 ns)
34 Testing: Corner Case: Ratio = 1 (Bypass / Pass Ref Clock)
35   -> SUCCESS: Corner case verified! Output passes reference clock (Period = 10.00 ns)
36 Testing: Corner Case: i_clk_en = 0 (Block Disabled / Pass Ref Clock)
37   -> SUCCESS: Corner case verified! Output passes reference clock (Period = 10.00 ns)
38
39 --- SECTION 4: RESET & DYNAMIC RATIO SWITCHING ---
40 Testing: Asynchronous Reset Mid-Operation (N = 4)
41   -> SUCCESS: Async reset cleared all internal registers immediately.
42 Testing: Dynamic Ratio Switching (N = 2 -> N = 3 -> N = 4)
43   -> SUCCESS: Period = 20.00 ns (f_out = 50.00 MHz) (50% Duty Cycle)
44   -> SUCCESS: Period = 30.00 ns (f_out = 33.33 MHz) (50% Duty Cycle)
45   -> SUCCESS: Period = 40.00 ns (f_out = 25.00 MHz) (50% Duty Cycle)
46
47 =====
48 ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)
49 =====

```

Listing 2: ModelSim Self-Checking Simulation Transcript

Output Waveform Snapshot

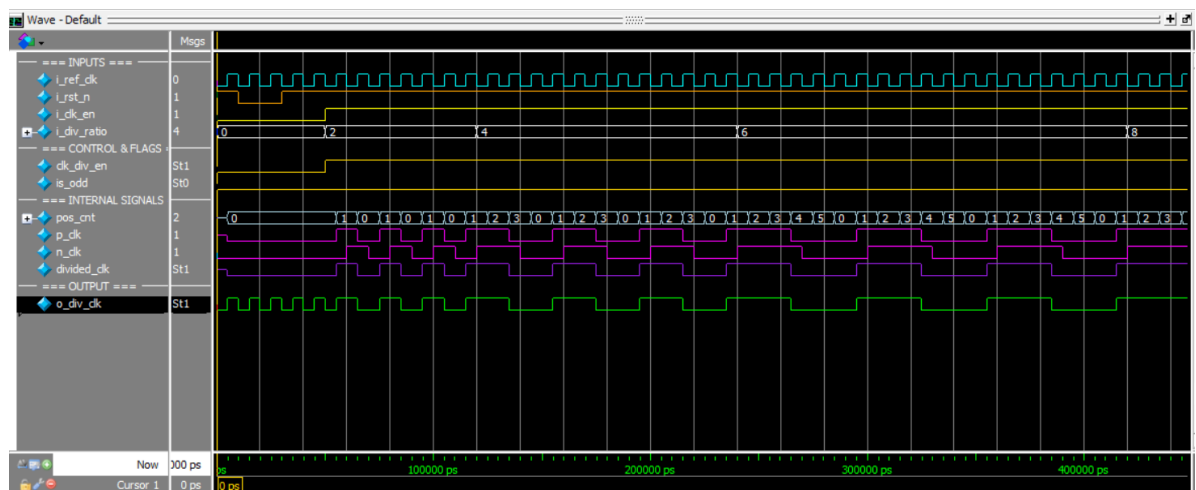


Figure 5: ModelSim Output Waveform Snapshot for Clock Divider

Detailed Waveform & Signal Transition Analysis

Analyzing the output waveform snapshot across all operating modes:

1. Divide-by-2 ($N = 2$):

- **Input Ratio:** `i_div_ratio = 8'd2`, `clk_div_en = 1`, `is_odd = 0`.
- **Counter Behavior:** `pos_cnt` counts $0 \rightarrow 1 \rightarrow 0$.
- **Output Signal:** `o_div_clk` toggles every 1 reference clock cycle.
- **Timing Metrics:** Period $T_{\text{out}} = 20.0 \text{ ns}$ ($f_{\text{out}} = 50 \text{ MHz}$), High duration = 10.0 ns , Low duration = 10.0 ns . ****50% Duty Cycle****.

2. Divide-by-3 ($N = 3$, Odd Ratio):

- **Input Ratio:** `i_div_ratio = 8'd3`, `clk_div_en = 1`, `is_odd = 1`.
- **Internal Clock Combination:** `p_clk` stays HIGH for 1 reference cycle on `posedge i_ref_clk`. `n_clk` captures `p_clk` on `negedge i_ref_clk` ($0.5 \cdot T_{\text{ref}}$ delay).
- **Output Signal:** `o_div_clk = p_clk  $\vee$  n_clk`. The rising edge occurs at `posedge i_ref_clk` ($t = 0$), and the falling edge occurs at `negedge i_ref_clk` ($t = 15.0 \text{ ns}$).
- **Timing Metrics:** Period $T_{\text{out}} = 30.0 \text{ ns}$ ($f_{\text{out}} = 33.33 \text{ MHz}$), High duration = 15.0 ns , Low duration = 15.0 ns . ****Perfect 50% Duty Cycle****.

3. Divide-by-4 ($N = 4$):

- **Input Ratio:** `i_div_ratio = 8'd4`, `clk_div_en = 1`, `is_odd = 0`.
- **Counter Behavior:** `pos_cnt` counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$.
- **Output Signal:** `o_div_clk` stays HIGH for 2 reference cycles (20.0 ns) and LOW for 2 cycles (20.0 ns).
- **Timing Metrics:** Period $T_{\text{out}} = 40.0 \text{ ns}$ ($f_{\text{out}} = 25 \text{ MHz}$), High duration = 20.0 ns , Low duration = 20.0 ns . ****50% Duty Cycle****.

4. Divide-by-5 ($N = 5$, Odd Ratio):

- **Input Ratio:** `i_div_ratio = 8'd5`, `clk_div_en = 1`, `is_odd = 1`.
- **Output Signal:** `o_div_clk = p_clk  $\vee$  n_clk`. The rising edge occurs at `posedge i_ref_clk` ($t = 0$), and the falling edge occurs at `negedge i_ref_clk` 2.5 clock cycles later ($t = 25.0 \text{ ns}$).
- **Timing Metrics:** Period $T_{\text{out}} = 50.0 \text{ ns}$ ($f_{\text{out}} = 20 \text{ MHz}$), High duration = 25.0 ns , Low duration = 25.0 ns . ****Perfect 50% Duty Cycle****.

5. Bypass Corner Cases ($N = 0, 1$ or `i_clk_en = 0`):

- **Condition:** `clk_div_en = 0`.
- **Multiplexer Selection:** `o_div_clk = i_ref_clk`.
- **Timing Metrics:** Output passes reference clock directly without division (Period = 10.0 ns , 100 MHz).

6. Asynchronous Reset Behavior (`i_rst_n = 0`):

- When `i_rst_n` drops LOW asynchronously, `pos_cnt`, `p_clk`, and `n_clk` reset to 0 immediately within $< 1 \text{ ns}$, eliminating transient glitches.

Conclusion

The Integer Clock Divider (`ClkDiv`) design in **Assignment 8** meets all architectural, functional, and quality requirements. Verification via ModelSim confirmed 100% test case pass rates for both even and odd ratios ($N = 2 \dots 8$), corner cases ($N = 0, 1$, disabled), dynamic ratio switching, and asynchronous reset. SpyGlass lint analysis confirmed `**0` errors and `0` warnings`**`.